General Requirements

1. Use modern C++ (C++17 or later unless your instructor specifies otherwise).
2. Use appropriate encapsulation, abstraction, inheritance/composition, and polymorphism.
3. Use smart pointers when ownership is required. Raw pointers/references must be justified.
4. Avoid large switch/if-else chains when the scenario explicitly requires extensibility.
5. Do not select a pattern based only on method or class names. Explain the responsibility and intent.
6. Your program must compile and include a small main() demonstration or automated tests.
7. For each case, include a short design explanation (approximately 150-300 words).

Patterns in Scope

Builder, Strategy, Prototype, Registry, Factory. A case may use one, several, or none of these in a particular subsystem. You must justify your choices.

Submission Package

- UML class diagram (image or PDF).
- C++ source files (.h/.hpp and .cpp as appropriate).
- README containing build/run instructions.
- Design explanation and answers to the discussion questions.
- Test evidence: console output.

Evaluation Checklist

Area	What will be checked	Weight
Design reasoning	Pattern intent, separation of responsibilities, extensibility	30%
C++ implementation	Correctness, polymorphism, STL use, compilation	30%
Ownership & copying	Smart pointers, lifetime, deep/shallow copy decisions	20%
Testing	Normal, edge, and failure cases	10%
Clarity	UML, naming, readability, explanation	10%

Design defense
Be prepared to explain why your design uses a pattern. A solution that only renames classes to Builder, Factory, Strategy, etc. without matching the pattern responsibility will not receive full credit.

Case Study 1 - FlySmart International Travel Platform

FlySmart is an online travel company that allows customers to build personalized international travel packages. A few years ago most customers simply booked a flight. Today, travelers expect a flexible combination of flights, hotels, baggage, insurance, transfers, meals, and tours.

Linh is traveling to Tokyo for three days and wants an economy flight, a small hotel, and one checked bag. **David** is traveling to Paris for his honeymoon and wants business class, a five-star hotel, airport pickup, travel insurance, special meals, and a private city tour.

The Current Problem

The existing system creates packages using a long constructor:

```
TravelPackage(
    "Ho Chi Minh City",
    "Paris",
    "Business",
    "Grand Hotel",
    5,
    true,
    2,
    "Window",
    true,
    true,
    true
);
```

As options were added, developers began mixing up boolean and integer parameters. The team wants client code that is readable and supports optional configuration.

Required Client Experience

```
auto trip = TravelPackageBuilder()
    .from("Ho Chi Minh City")
    .to("Paris")
    .flightClass("Business")
    .hotel("Grand Hotel")
    .nights(5)
    .checkedBags(2)
    .seat("Window")
    .airportPickup()
    .travelInsurance()
    .privateTour()
    .build();
```

Functional Requirements

- Required: departure city, destination, and flight class.
- Optional: hotel, nights, baggage, seat, meals, airport pickup, insurance, city transport, and local tours.
- Number of nights cannot be negative.
- Checked baggage cannot be negative.
- A private airport limousine is available only for Business or First Class.
- If a hotel is selected, the number of nights must be greater than zero.
- Departure and destination cannot be the same.

Change Request: Reusing the Construction Object

```
TravelPackageBuilder builder;
```

```
auto honeymoon = builder
    .from("Ho Chi Minh City")
    .to("Paris")
    .flightClass("Business")
    .hotel("Grand Hotel")
    .nights(5)
    .airportPickup()
    .travelInsurance()
    .privateTour()
    .build();

auto businessTrip = builder
    .from("Ho Chi Minh City")
    .to("Singapore")
    .flightClass("Economy")
    .hotel("City Hotel")
    .nights(2)
    .build();
```

During testing, the second package unexpectedly includes airport pickup, insurance, and a private tour. Your design must explain and prevent this state-leakage problem.

Change Request: Standard Packages

- Budget Traveler: Economy flight, one checked bag, standard hotel.
- Business Traveler: Business-class flight, airport pickup, business hotel, travel insurance.
- Luxury Traveler: First-class flight, five-star hotel, airport limousine, travel insurance, private tour, premium meals.

Marketing wants these presets created consistently without duplicating the same configuration code throughout the application.

Deliverables for Case 1

8. UML class diagram.
9. C++ implementation and demonstration.
10. At least three package examples.
11. Validation and error handling.
12. Demonstration of the state-reuse bug and your correction.
13. A short explanation of the design pattern(s) you selected.

Discussion Questions

14. Is inheritance necessary for your construction design? Why or why not?
15. Does your solution require a separate Director-like object? Defend your choice.
16. Who should perform validation: the product, the construction object, or both?
17. After build(), should the construction object reset, become unusable, or remain reusable? Explain the trade-offs.
18. If build() returns std::move(package), what happens to the internal object?

Case Study 2 - AurumTech Gold Trading Platform

AurumTech builds software for investors who trade gold. The system receives a continuous stream of prices. AurumTech does not want the trading engine itself to decide whether an investor should buy, sell, or hold; different clients use different decision rules.

```
09:00  3350
09:01  3353
09:02  3348
09:03  3360
...
```

Investor Behaviors

- Conservative investor: considers buying when price is significantly below a recent average.
- Momentum trader: reacts to sustained upward or downward movement.
- Threshold trader: uses configurable buy and sell price thresholds.

The Current Problem

```
class GoldTradingSystem {
public:
    void analyze(double price, std::string strategy) {
        if (strategy == "conservative") {
            // ...
        } else if (strategy == "momentum") {
            // ...
        } else if (strategy == "threshold") {
            // ...
        }
    }
};
```

Six months later the company supports many algorithms. The conditional chain is large, difficult to test, and must be edited whenever a new algorithm is added.

Requirement A: Replaceable Trading Behavior

A trading bot must be able to change its decision behavior at runtime without modifying the bot class for every new algorithm.

```
GoldTradingBot bot;

bot.setTradingStrategy(
    std::make_unique<MomentumStrategy>()
);
bot.analyze(3350);

bot.setTradingStrategy(
    std::make_unique<ConservativeStrategy>()
);
bot.analyze(3200);
```

Requirement B: Stateful Algorithms

Some algorithms store history:

```
class MomentumStrategy {
    std::vector<double> history;
public:
    Decision analyze(double price);
};
```

Decide whether two bots should share one strategy object or own independent objects. Your answer must discuss how state changes the decision.

Requirement C: Runtime Registration

AurumTech wants third-party developers to add algorithms without modifying a central if/else chain. A plugin should be able to register a creator:

```
registry.registerStrategy(
    "ai_trend",
    [] {
        return std::make_unique<AITrendStrategy>();
    }
);

auto strategy = registry.create("ai_trend");
```

Registry API

- registerStrategy(name, creator)
- create(name)
- contains(name)
- remove(name)
- safe behavior for unknown names

Architecture Comparison

Compare these two possible registries:

```
// A: stores creators
std::unordered_map<
    std::string,
    std::function<std::unique_ptr<TradingStrategy>()>
> creators;

// B: stores configured objects
std::unordered_map<
    std::string,
    std::unique_ptr<TradingStrategy>
> objects;
```

If version B creates a new object by calling clone() on the stored object, is it conceptually the same creation mechanism as version A? Explain based on what is stored and how a new instance is produced.

Lifetime Challenge

```
GoldTradingBot createBot() {
    MomentumStrategy strategy;
    GoldTradingBot bot(&strategy);
    return bot;
}
```

Assume the bot stores TradingStrategy*. Explain the lifetime problem and propose a safer ownership model.

Deliverables for Case 2

19. UML class diagram.
20. Trading behavior abstraction plus at least three concrete algorithms.
21. GoldTradingBot with runtime behavior replacement.
22. Runtime registration and dynamic creation by name.
23. Safe unknown-key handling.

24. Ownership/lifetime explanation.
25. At least five tests, including a stateful algorithm case.

Discussion Questions

26. What responsibility belongs to the trading bot, and what responsibility belongs to the trading algorithm?
27. What responsibility belongs to the registry?
28. Is inheritance mandatory for interchangeable algorithms in modern C++? Discuss std::function or templates as alternatives.
29. Does a method named create() automatically mean a Factory pattern is present? Why or why not?
30. When would sharing a stateful strategy be intentional?

Case Study 3 - VoltEdge Electric Vehicle Platform

VoltEdge Motors began with one electric city car. It now sells city, sport, SUV, taxi, delivery, and performance models. Customers configure vehicles online, while taxi fleets, delivery companies, rental firms, and partner plugins need reusable configurations and extensible creation.

Your team must redesign the platform so that object construction, driving behavior, cloning, lookup, and new vehicle creation do not collapse into one giant class.

Chapter 1: Configuring a Vehicle

A vehicle may contain model, battery, motor, drive system, wheel size, interior, paint, driver-assistance package, charging package, performance package, winter package, and other options.

```
ElectricCar(
    "VoltEdge SUV", 100, "Dual Motor", "AWD", 21,
    "Premium", "Black", true, true, true,
    false, false, true
);
```

The client code must become readable, support optional configuration, and enforce rules.

```
auto car = ElectricCarBuilder()
    .model("VoltEdge SUV")
    .battery(100)
    .motor("Dual Motor")
    .drive("AWD")
    .wheels(21)
    .interior("Premium")
    .paint("Black")
    .autopilot()
    .fastCharging()
    .build();
```

- Required: model, battery, and motor.
- Performance Mode requires an appropriate high-performance motor configuration.
- AWD requires a compatible multi-motor configuration.
- High-power charging requires a compatible battery.
- A two-seat sport configuration cannot be combined with a taxi package.

Chapter 2: Driving Modes

The same car can behave differently in Eco, Comfort, Sport, Snow, Track, or Autonomous mode. New driving algorithms must be addable without editing ElectricCar for every mode.

```
car.setDrivingMode(
    std::make_unique<EcoMode>()
);

// later
car.setDrivingMode(
    std::make_unique<SportMode>()
);
```

Discuss ownership and whether sharing a stateful driving-mode object between cars is safe or intentional.

Chapter 3: Fleet Templates

A taxi customer orders thousands of nearly identical vehicles. VoltEdge stores one fully configured template under a name such as "city_taxi_standard" and creates independent cars from it.

```
auto taxi1 = fleetRegistry.create("city_taxi_standard");
auto taxi2 = fleetRegistry.create("city_taxi_standard");
```

Changing taxi1 must not unexpectedly modify taxi2 or the stored template.

Chapter 4: Deep Copy Challenge

```
class ElectricCar {
    std::unique_ptr<Battery> battery;
    std::unique_ptr<Motor> motor;
    std::unique_ptr<DrivingMode> mode;
};

std::unique_ptr<ElectricCar> clone() const {
    return std::make_unique<ElectricCar>(*this);
}
```

The clone implementation does not compile. Explain why and design a correct independent-copy solution. Decide which internal components require their own cloning/copying behavior.

Chapter 5: Fleet Registry

Saved configurations include names such as city_taxi_standard, airport_shuttle, cold_weather_delivery, and police_patrol. The caller should create by key without knowing where the template is stored.

- add(...)
- remove(...)
- contains(...)
- create(...)
- safe unknown-key behavior

Chapter 6: Registry Lookup Trap

```
std::unordered_map<
    std::string,
    std::unique_ptr<ElectricCar>
> registry;

auto& prototype = registry["police_car"];
```

Assume "police_car" was never registered. Explain what operator[] does and compare it with find() and at() for lookup-only operations.

Chapter 7: Partner Vehicle Plugins

A partner company creates a new RefrigeratedDeliveryVan type. The core system should not add another if/else branch every time a partner adds a vehicle.

```
vehicleFactory.registerCreator(
    "refrigerated_delivery",
    [] {
        return std::make_unique<RefrigeratedDeliveryVan>();
    }
);

auto van = vehicleFactory.create("refrigerated_delivery");
```

Chapter 8: Two Similar-Looking Creation Systems

```
// System A
std::unordered_map<
    std::string,
```

```
    std::unique_ptr<ElectricCar>
> prototypes;

return prototypes.at(name)->clone();

// System B
std::unordered_map<
    std::string,
    std::function<std::unique_ptr<ElectricCar>()>
> creators;

return creators.at(name)();
```

Are these systems using the same creation mechanism? Your explanation must describe what is stored and how the new object is produced. Do not classify them based only on register(), create(), or class names.

Chapter 9: The Factory-Method Naming Trap

```
class VehicleFactory {
public:
    std::unique_ptr<ElectricCar>
    create(std::string type);
};
```

A teammate claims this must be the Factory Method design pattern because create() is a method in a class named VehicleFactory. Evaluate the claim using pattern intent and structure.

Final Integration Scenario

```
VehicleFactory factory;
factory.registerCreator("sport", [] {
    return std::make_unique<SportEV>();
});

auto car = ElectricCarBuilder()
    .model("VoltEdge Performance")
    .battery(100)
    .motor("Dual Motor")
    .drive("AWD")
    .performancePackage()
    .build();

car.setDrivingMode(std::make_unique<SportMode>());

FleetRegistry fleets;
fleets.add("airport_shuttle", configuredShuttle);

auto shuttle1 = fleets.create("airport_shuttle");
auto shuttle2 = fleets.create("airport_shuttle");

shuttle1->setDrivingMode(std::make_unique<EcoMode>());
```

The system must keep shuttle1, shuttle2, and the stored template independent unless sharing is explicitly designed and justified.

Deliverables for Case 3

31. Complete UML class diagram showing ownership and important relationships.
32. C++ implementation of the major subsystems.
33. Complex vehicle configuration with validation.
34. Runtime replaceable driving behavior.
35. Independent creation from saved fleet templates.

36. Named registry operations and safe lookup.
37. Extensible vehicle creation through registration.
38. At least eight tests, including invalid configuration, unknown keys, cloning, and runtime behavior changes.
39. A 250-500 word architecture explanation that identifies and justifies the patterns used.

Architecture Reasoning Questions

40. Which subsystem handles complicated object construction, and why?
41. Which subsystem handles interchangeable driving behavior, and why?
42. Which subsystem creates an object from an existing configured object?
43. Which subsystem maps names to saved configurations?
44. Which subsystem hides which concrete vehicle type is instantiated?
45. Can one class participate in more than one design pattern? Give an example from your design.
46. Is a Registry necessarily a Factory? Explain.
47. Is a Factory necessarily implemented using inheritance? Explain.
48. Does every class with clone() correctly implement Prototype? Explain.
49. Does every function returning std::unique_ptr<Base> represent a Factory? Explain.

Suggested File Naming

```
studentID_case1/
studentID_case2/
studentID_case3/

Each folder:
  README.md
  diagram.pdf (or diagram.png)
  include/
  src/
  tests/
```

End of requirement document